DP-OPS-0069: Comprehensive Migration Strategy for Case-Normalization of Canon Artifacts and Registry Identifiers

1. Governance Framework and Architectural Context

1.1 The Constitutional Foundation: Policy of Truth

The stela-platform repository operates under a governance model that is more akin to a legal jurisdiction than a standard software project. At the apex of this structure stands the **Policy of Truth (PoT.md)**, a document explicitly identified in the repository's "Read-in Order" as the primary source of truth.[1] This constitutional document dictates the "Physical Laws" of the environment, establishing a rigid separation of concerns known as the **Filing Doctrine**.[1]

The Filing Doctrine partitions the repository into functional zones, each with distinct semantic roles and behavioral expectations:

- **ops/ (Run):** The operational kernel containing executable logic, binaries, and manifests. This layer is responsible for the mechanical reproduction of the system's state.
- **docs/ (Explain):** The explanatory layer containing manuals, specifications, and rationale. This layer provides the "why" behind the "how."
- **opt/ (Isolate):** A quarantine zone for JIT-only (Just-In-Time) content, specifically the Factory definitions (_factory), which are excluded from the global context to prevent token pollution in the LLM context window.
- **storage/ (Payload):** A designated area for transient artifacts, handoffs, and dumps.

Within this framework, file naming conventions are not merely aesthetic choices; they are semantic markers that indicate the stability and authority of a file. The current state of the repository reveals a "Case Schism" that DP-OPS-0069 seeks to resolve. While the ops/bin directory utilizes strictly lowercase executables (compile, context, draft, dump) [1], the opt/_factory and docs/ops/registry directories currently utilize Uppercase filenames (R-AGENT-01.md, AGENTS.md). This inconsistency creates cognitive dissonance, complicates tooling regexes, and increases the risk of cross-platform filesystem collisions.

The proposed migration is a constitutional amendment to the Filing Doctrine. It aligns the *instances* of work units (specific agents, tasks, skills) with the lowercase convention of the *tools* that manipulate them, while reserving Uppercase exclusively for *Classes* (Registries), *Indices* (Roots), and *Constitution* (PoT, SoP, TASK). This establishes a visual hierarchy where Uppercase files denote immutable or high-level structures, and lowercase files denote

mutable, executable instances.

1.2 The "Quartet" Topology Alignment

The repository recently underwent a significant structural shift documented in **DP-OPS-0068**, known as the "Quartet Filing Doctrine and Topology Alignment".[1] This operation redefined the storage layer and introduced new root directories such as var/tmp for resume states, logs for telemetry, and archives for cold storage. This topology shift was critical in establishing a "clean tree" philosophy, ensuring that transient files do not pollute the canonical history.

DP-OPS-0069 is the logical successor to the Quartet alignment. Having stabilized the *directories*, the focus now shifts to the *files* within them. The migration to lowercase identifiers (r-agent-, b-task-) for instance files harmonizes the repository with standard POSIX conventions, reducing friction for automation tools that typically default to lowercase processing. It also resolves a long-standing tension between the "human-readable" Uppercase IDs used in documentation and the "machine-readable" lowercase paths preferred by the ops/bin toolchain.

1.3 Hard Constraints and Preservation Rules

Any modification to the stela-platform must strictly adhere to the "No Invention" rule codified in the Copilot Instructions.[1] This mandate forbids the creation of repository structures or paths not explicitly defined in the PoT.md. Therefore, the migration plan must be rigorous in its scope definition, ensuring that it normalizes existing paths rather than inventing new architectures.

Crucially, the user query imposes a set of **Hard Preservation Constraints**. These constraints are non-negotiable and serve to maintain the "Explain" layer's visibility and the "Run" layer's integrity.

1.3.1 The Preservation of Factory Roots

The directory opt/_factory/ acts as the generative heart of the repository. It contains the definitions for Agents, Skills, and Tasks. While the *instances* (e.g., R-AGENT-01.md) are targets for lowercasing, the **Root Indices** must remain Uppercase [1]:

- **opt/_factory/AGENTS.md**: The master ledger for all agents.
- **opt/_factory/SKILLS.md**: The master ledger for all skills.
- **opt/_factory/TASKS.md**: The master ledger for all tasks.
- **opt/_factory/INDEX.md**: The navigational anchor for the factory.

Retaining these as Uppercase creates a clear visual distinction between the "Manager" files (Uppercase) and the "Managed" files (lowercase). It prevents the root directories from becoming a visual soup of uniform filenames, allowing operators to instantly identify the

controlling surfaces.

1.3.2 The Preservation of Generated Manifests

The directory ops/lib/manifests/ contains the authoritative state definitions for the system's context window. These files—CONTEXT.md, CORE.md, OPS.md, DISCOVERY.md, and CONSTRAINTS.md—are generated artifacts produced by the ops/bin/compile binary.[1]

These files must remain Uppercase for two reasons:

1. **Semantic Weight:** They represent the "Laws" of the current session. CORE.md defines the physical laws; OPS.md defines the operational capabilities. Uppercase naming signifies their immutability during a session.
2. **Tooling Contract:** The ops/bin/compile script likely contains hardcoded logic that maps lowercase templates (e.g., ops/src/manifests/core.md.tpl) to Uppercase outputs. Altering this output casing would require a deep refactor of the binary's internal logic, which poses a high risk of regression for a "one-off" migration. Preservation ensures continuity of the compilation contract.

1.3.3 The Preservation of Documentation Anchors

The files README.md and INDEX.md are standard waypoints in the GitHub ecosystem and the Stela documentation topology.

- **README.md**: The entry point for any directory.
- **INDEX.md**: A curated list of contents, distinct from a simple file listing.

Converting these to lowercase (readme.md) would reduce their visual prominence in file explorers and violate the established convention that "Navigational Aids are Uppercase." The migration must explicitly exclude these files from any bulk-rename operations to maintain the "Explain" layer's accessibility.[1]

---

2. Technical Constraints: The Physics of Git and Filesystems

2.1 The Divergence of Filesystem Architectures

A primary technical driver for this migration is the inherent danger of mixed-case filenames in a distributed version control system used across heterogeneous operating systems. The research material highlights significant risks associated with the disparate handling of case sensitivity between Windows/macOS and Linux environments.[2]

- **Linux (Ext4/XFS):** These filesystems are natively **case-sensitive**. To a Linux kernel, R-AGENT-01.md and r-agent-01.md are two completely distinct files, occupying different inodes. They can coexist in the same directory without conflict.

- **macOS (APFS) / Windows (NTFS):** These filesystems are **case-insensitive but case-preserving**. They remember that a file was named R-AGENT-01.md, but they treat r-agent-01.md as a synonym for the same file path. You cannot have both files in the same directory; the OS considers it a collision.[5]

This divergence creates a critical vulnerability in Git repositories shared across teams using different OSs. If a developer on Linux creates r-agent-01.md while R-AGENT-01.md already exists, they successfully commit two files. When a developer on macOS attempts to pull this commit, Git fails to checkout the second file because the filesystem reports a collision, leading to "ghost file" scenarios where the working tree is perpetually dirty or incomplete.[7]

2.2 The core.ignorecase Configuration Hazard

Git attempts to mitigate this filesystem divergence using the core.ignorecase configuration setting.

- On **Windows/macOS**, git init or git clone typically probes the filesystem and sets core.ignorecase = true by default.[3]
- On **Linux**, it defaults to false.

When core.ignorecase is true, Git does not detect a simple case change as a modification. If an operator renames AGENTS.md to agents.md using the operating system's file explorer, Git sees no change in the working tree. The file on disk is lowercase, but the entry in the Git index remains Uppercase. This synchronization gap is the root cause of "index drift".[9]

If the operator then forces the update (e.g., via git add), the repository enters a "Split-Brain" state. The index might record the file as agents.md, but older commits or other branches might still reference AGENTS.md. Merging these branches becomes a nightmare of spurious conflicts where Git thinks the file has been deleted and recreated, rather than renamed.

2.3 The "Split-Brain" Index Risk

The most severe manifestation of this issue is the **Split-Brain Index**.[11] This occurs when the Git index contains *both* the Uppercase and lowercase versions of the file, while the underlying filesystem can only support one.

1. **State:** Repo contains File.md.
2. **Action:** Operator renames to file.md improperly.
3. **Result:** Git index records File.md (from history) and file.md (new add).
4. **Consequence:** On Linux, git checkout produces two files. On Mac, git checkout overwrites one with the other, leading to "modified content" warnings immediately after a clean checkout.

This state is "toxic" for a repository governed by strict auditing rules like Stela's PoW.md (Proof of Work), which relies on exact file paths for hash verification. A Split-Brain index

renders the Proof of Work invalid because the file paths are ambiguous.

2.4 The Atomic Rename Protocol (Mitigation Strategy)

To execute DP-OPS-0069 safely, the Operator must utilize a rigorous **Atomic Rename Protocol**.[3] This protocol bypasses the core.ignorecase limitations by ensuring that every rename operation is registered as a distinct "Move" rather than a "Modification."

The Protocol:

1. **Intermediate Move:** Rename the source file (FILE) to a temporary hash-collided name (FILE_tmp). This forces Git to register a delete operation for the original filename, regardless of case sensitivity settings.
2. **Final Move:** Rename the temporary file (FILE_tmp) to the target lowercase name (file). This forces Git to register a create operation for the new filename.

By breaking the rename into two transactions, we force the Git index to flush the old inode reference and generate a new one, guaranteeing consistency across all operating systems. This protocol must be scripted into the migration logic to prevent human error.

---

3. Repository Inventory and State Assessment

A precise inventory of the repository's current state is required to define the scope of the migration. Based on the Dump Manifest [1] and the Dump Payload [1], the following surfaces are identified as targets.

3.1 Target Group A: Operational Prompts (docs/ops/prompts/)

This directory contains the "Stance" definitions for the AI operators.

- **Current Pattern:** E-PROMPT-XX.md (Uppercase)
- **Target Pattern:** e-prompt-xx.md (Lowercase)
- **Inventory:**
  - docs/ops/prompts/E-PROMPT-01.md
  - docs/ops/prompts/E-PROMPT-02.md
  - docs/ops/prompts/E-PROMPT-03.md
  - docs/ops/prompts/E-PROMPT-04.md
- **Exclusion:** docs/ops/prompts/README.md **(PRESERVE UPPERCASE)**.[1] This file explains the prompt directory and must remain a standard anchor.

3.2 Target Group B: Operational Registry (docs/ops/registry/)

This directory acts as the central database for all system components. Lowercasing these files aligns them with the ops/bin executables they document.

- **Current Pattern:** .md (Uppercase)
- **Target Pattern:** [category].md (Lowercase)
- **Inventory:**
  - AGENTS.md -> agents.md
  - BINARIES.md -> binaries.md
  - LINT.md -> lint.md
  - PROJECTS.md -> projects.md
  - PROMPTS.md -> prompts.md
  - SCRIPTS.md -> scripts.md
  - SKILLS.md -> skills.md
  - TASKS.md -> tasks.md
  - TEST.md -> test.md
  - TOOLS.md -> tools.md
- **Note:** No README.md or INDEX.md exists in this directory [1], simplifying the operation.

3.3 Target Group C: Factory Agents (opt/_factory/agents/)

These are the definition files for the autonomous agents.

- **Current Pattern:** R-AGENT-XX.md (Uppercase)
- **Target Pattern:** r-agent-xx.md (Lowercase)
- **Inventory:**
  - opt/_factory/agents/R-AGENT-01.md
  - opt/_factory/agents/R-AGENT-02.md
  - opt/_factory/agents/R-AGENT-03.md
  - opt/_factory/agents/R-AGENT-04.md
  - opt/_factory/agents/R-AGENT-05.md
  - opt/_factory/agents/R-AGENT-06.md

3.4 Target Group D: Factory Skills (opt/_factory/skills/)

These are the executable knowledge units.

- **Current Pattern:** S-LEARN-XX.md (Uppercase)
- **Target Pattern:** s-learn-xx.md (Lowercase)
- **Inventory:**
  - opt/_factory/skills/S-LEARN-01.md
  - opt/_factory/skills/S-LEARN-02.md
  - opt/_factory/skills/S-LEARN-03.md
  - opt/_factory/skills/S-LEARN-04.md
  - opt/_factory/skills/S-LEARN-05.md
  - opt/_factory/skills/S-LEARN-06.md

3.5 Target Group E: Factory Tasks (opt/_factory/tasks/)

These are the standard operating procedures.

- **Current Pattern:** B-TASK-XX.md (Uppercase)
- **Target Pattern:** b-task-xx.md (Lowercase)
- **Inventory:**
  - opt/_factory/tasks/B-TASK-01.md
  - opt/_factory/tasks/B-TASK-02.md
  - opt/_factory/tasks/B-TASK-03.md
  - opt/_factory/tasks/B-TASK-04.md
  - opt/_factory/tasks/B-TASK-05.md
  - opt/_factory/tasks/B-TASK-06.md

3.6 Preservation Group (Strict Exclusion)

The following files are strictly excluded from the migration script. They serve as the "Fixed Stars" of the repository topology.

- **Factory Roots:**
  - opt/_factory/AGENTS.md
  - opt/_factory/SKILLS.md
  - opt/_factory/TASKS.md
  - opt/_factory/INDEX.md
- **Manifests:**
  - ops/lib/manifests/CONSTRAINTS.md
  - ops/lib/manifests/CONTEXT.md
  - ops/lib/manifests/CORE.md
  - ops/lib/manifests/DISCOVERY.md
  - ops/lib/manifests/OPS.md
- **Canon:** PoT.md, SoP.md, TASK.md, PoW.md.
- **Documentation:** docs/MANUAL.md, docs/MAP.md, docs/INDEX.md, README.md.

---

4. Tooling Impact Analysis and Remediation Plan

The repository relies on a sophisticated chain of tooling—binaries, harvesters, and linters—that enforces the current Uppercase convention. Renaming files without simultaneously updating these tools will trigger catastrophic failure in the CI/CD pipeline (Repo Gates).[1]

4.1 The Compilation Pipeline (ops/bin/compile)

The ops/bin/compile binary is responsible for aggregating templates into manifests.

- **Current Behavior:** It reads lowercase templates (ops/src/manifests/*.tpl) and generates Uppercase manifests (ops/lib/manifests/*.md).[1]

- **Impact:** The migration preserves the *output filenames* of the manifests (they stay Uppercase). However, the *content* of those manifests is derived from the templates.
- **Remediation:** The templates themselves (ops/src/manifests/*.tpl) must be updated to reference the new lowercase paths. For example, ops/src/manifests/ops.md.tpl currently lists docs/ops/registry/AGENTS.md. This must be changed to docs/ops/registry/agents.md. The binary itself does not need a filename change, but its internal logic might filter for Uppercase files. Given the lack of source code in the dump, we must assume it scans for explicit paths defined in templates.

4.2 The Harvester Scripts (ops/lib/scripts/*)

The scripts agent.sh, task.sh, and skill.sh manage the lifecycle of factory items.

- **Current Logic:** They generate IDs like R-AGENT-XX and filenames like R-AGENT-XX.md. They likely use printf formatting or tr commands to enforce uppercase.[1]
- **Remediation:**
  - **ID Generation:** Update the logic to produce r-agent-xx (lowercase). This involves changing printf 'R-AGENT-%s' to printf 'r-agent-%s' and potentially removing or reversing tr '[:lower:]' '[:upper:]' pipes.
  - **Filename Generation:** Update the output path construction to match the lowercase ID.
  - **Registry Updates:** These scripts automatically append entries to the Registries (AGENTS.md). They must be updated to target the new lowercase registry files (docs/ops/registry/agents.md) while still appending to the Uppercase Factory Roots (opt/_factory/AGENTS.md). This distinction is critical: the *global registry* moves to lowercase, but the *factory root ledger* stays uppercase.

4.3 The "Immune System" (Linters)

The tools/lint/ directory contains the "Factory Guard" (factory.sh) and other validators.

- **tools/lint/factory.sh:** This script enforces the integrity of the factory. It currently expects R-AGENT-*.md.
  - **Update:** Modify regex patterns to match r-agent-*.md.
  - **Update:** Point it to the new registry location (docs/ops/registry/agents.md).
- **tools/lint/truth.sh:** Validates "Authored Surfaces." It must be updated to accept e-prompt- and other lowercase IDs as valid canon references.
- **tools/lint/dp.sh:** Validates TASK.md. The active TASK.md will be updated to reference the new IDs, so the linter must support this format.

4.4 The "Project" Helper (ops/lib/scripts/project.sh)

This script validates project IDs. While projects are out of scope for this migration, ensuring that the shared libraries used by ops/bin/project do not enforce uppercase constraints on global resources is a necessary check.

---

5. Migration Execution Protocol (DP-OPS-0069)

This section defines the executable procedure for the Operator. It is structured as a sequence of atomic operations to ensure recoverability at any stage.

5.1 Phase 1: Tooling Pre-Flight (The "Green Build" Prep)

Before moving a single file, the tooling must be patched to support the destination state. This prevents the CI system from rejecting the changes during the PR process.

Step 1.1: Update Harvester Generators

Edit ops/lib/scripts/agent.sh, task.sh, and skill.sh.

- Locate ID generation logic (e.g., next_id=$(...)).
- Change prefix literals from R-AGENT- to r-agent-.
- Change file path construction to use lowercase string formatting.

Step 1.2: Update Linters

Edit tools/lint/factory.sh and tools/lint/agent.sh.

- Update regex validation from [A-Z]-[A-Z]+-[0-9]+ to [a-z]-[a-z]+-[0-9]+.
- Update target paths to docs/ops/registry/agents.md (etc.).

Step 1.3: Update Manifest Templates

Edit ops/src/manifests/*.tpl.

- Perform a global replacement of the target uppercase paths with their lowercase equivalents.
- Example: docs/ops/registry/BINARIES.md -> docs/ops/registry/binaries.md.

5.2 Phase 2: The Atomic Rename (Filesystem)

This phase executes the filesystem changes using the **Atomic Rename Protocol**. A shell script migrate_casing.sh should be generated and run.

Target: Prompts

Bash

```
# Rename E-PROMPT files
```

```
for f in docs/ops/prompts/E-PROMPT-*.md; do
    target=$(echo "$f" | tr '[:upper:]' '[:lower:]')
    git mv "$f" "${f}.tmp"
    git mv "${f}.tmp" "$target"
done
# Verification: Ensure README.md was NOT touched.
```

Target: Registries

```
    Bash
```

```
# Rename Registry files (excluding non-targets)
cd docs/ops/registry
for f in *.md; do
    if]; then continue; fi
    target=$(echo "$f" | tr '[:upper:]' '[:lower:]')
    git mv "$f" "${f}.tmp"
    git mv "${f}.tmp" "$target"
done
cd../../..
```

Target: Factory Definitions

```
    Bash
```

```
# Rename Agents, Skills, Tasks
for type in agents skills tasks; do
    for f in opt/_factory/"$type"/*.md; do
        # Extract filename
        base=$(basename "$f")
        dir=$(dirname "$f")
        # Lowercase the filename only
        lower_base=$(echo "$base" | tr '[:upper:]' '[:lower:]')
        target="$dir/$lower_base"

        git mv "$f" "${f}.tmp"
        git mv "${f}.tmp" "$target"
```

```
  done
done
```

5.3 Phase 3: Content ID Alignment (The "sed" Pass)

Renaming the files breaks every internal pointer that references them by their old Uppercase ID. We must execute a global search-and-replace to realign the content with the new filenames.

Scope of sed:

- **Target Directories:** ops/, docs/, opt/, projects/.
- **Excluded Directories:** archives/ (preserve history), .git/.
- **Excluded Files:** PoW.md (Proof of Work ledger must remain immutable), SoP.md (History ledger).

Replacement Logic:

1. **Update ID Prefixes:**
   - s/R-AGENT-/r-agent-/g
   - s/B-TASK-/b-task-/g
   - s/S-LEARN-/s-learn-/g
   - s/E-PROMPT-/e-prompt-/g
2. **Update Registry Paths:**
   - s/docs\/ops\/registry\/AGENTS.md/docs\/ops\/registry\/agents.md/g
   - (Repeat for all 10 registry files).

Special Case: The Factory Roots

The files opt/_factory/AGENTS.md (etc.) are preserved in Uppercase, but they contain lists of the agents. These lists must be updated to reflect the new lowercase IDs and filenames. The sed pass will handle this naturally if included in the target scope.

5.4 Phase 4: Regeneration and Verification

Once the files are moved and the content is updated, we must regenerate the derived artifacts to ensure the system is consistent.

Step 4.1: Compile Manifests

Run ./ops/bin/compile.

- This uses the updated templates (Phase 1) to regenerate ops/lib/manifests/OPS.md (etc.).
- **Verification:** Inspect ops/lib/manifests/OPS.md. It should be an Uppercase file containing a list of lowercase paths (docs/ops/registry/agents.md).

Step 4.2: Refresh Context

Run ./ops/bin/llms and ./ops/bin/map.

- This ensures the docs/MAP.md and llms.txt pointers are aligned with the new structure.

Step 4.3: Gate Check

Run the full suite of verification tools:

- ./tools/verify.sh
- bash tools/lint/truth.sh
- bash tools/lint/factory.sh

Any failure here indicates a missed reference or a broken regex in the tooling.

---

6. Risk Assessment and Mitigation Strategies

6.1 Documentation Drift

Risk: Links in docs/MANUAL.md or external documentation may still point to Uppercase paths (e.g., docs/ops/registry/AGENTS.md). GitHub handles some case-insensitivity in URLs, but not all.

Mitigation: The sed pass includes the docs/ directory. We explicitly target markdown links (path/to/FILE.md) to ensure they are lowercased.

6.2 The "Ghost File" Regression

Risk: If a developer has a local checkout with core.ignorecase=true and pulls these changes, they might end up with untracked files or conflicts if they have local modifications.

Mitigation: The deployment of this change should be accompanied by a "Flag Day" notification. Developers should be advised to run git clean -fdx after pulling the migration commit to flush any lingering filesystem ghosts.

6.3 Tooling Blindspots

Risk: If ops/bin/compile (binary) has hardcoded uppercase logic that was not visible in the templates, it might force the content of manifests back to uppercase during regeneration.

Mitigation: Post-execution verification of ops/lib/manifests/OPS.md is mandatory. If the compiler reverts the paths, the binary itself requires a patch (recompilation from source) or the template logic needs a workaround. Given the "No Invention" rule, we assume the binary follows the template's lead.

---

7. Conclusion

DP-OPS-0069 represents a critical maturity milestone for the stela-platform. By enforcing POSIX-compliant lowercase conventions for operational instances while preserving the Uppercase dignity of Constitutional and Root artifacts, the repository achieves a "Best of Both Worlds" state: high compatibility for tooling and high visibility for governance.

This migration resolves the "Case Schism," eliminates the "Split-Brain" index risk, and fortifies the Filing Doctrine against cross-platform entropy. The strict adherence to the Atomic Rename Protocol and the preservation of the ops/lib/manifests/ and opt/_factory/ roots ensures that this transition is non-destructive and fully reversible if required.

7.1 Final State Validation

Upon completion, the repository structure will exhibit the following consistent logic:

Component	**Casing**	**Rationale**
Constitution (PoT.md)	**UPPER**	Immutable Law
Factory Root (AGENTS.md)	**UPPER**	Class Definition / Index
Factory Instance (r-agent-01.md)	**lower**	Object Instance / Executable
Registry Index (agents.md)	**lower**	Operational Lookup
Tooling (ops/bin/compile)	**lower**	Executable Utility
Manifest (OPS.md)	**UPPER**	Generated Law

This hierarchy is linguistically precise, structurally sound, and technically robust.

Works cited

1. OPEN-main-f3d6b2e4.txt
2. Git case sensitivity - Azure Repos - Microsoft, accessed February 17, 2026, https://learn.microsoft.com/en-us/azure/devops/repos/git/case-sensitivity?view=azure-devops
3. How do I commit case-sensitive only filename changes in Git? - Stack Overflow,

accessed February 17, 2026, https://stackoverflow.com/questions/17683458/how-do-i-commit-case-sensitive-only-filename-changes-in-git
4. Fixing File Renaming Issues in Git: Handling Case Sensitivity and core.ignorecase, accessed February 17, 2026, https://dev.to/iediong/fixing-file-renaming-issues-in-git-handling-case-sensitivity-and-coreignorecase-1mf6
5. Git (MacOS) case sensitive overwrite issues - Stack Overflow, accessed February 17, 2026, https://stackoverflow.com/questions/25575463/git-macos-case-sensitive-overwrite-issues
6. Case-sensitive path collisions on case-insensitive file system when I do git clone, accessed February 17, 2026, https://stackoverflow.com/questions/63468346/case-sensitive-path-collisions-on-case-insensitive-file-system-when-i-do-git-clo
7. Git is case-sensitive and your filesystem may not be - Weird folder merging on Windows, accessed February 17, 2026, https://www.hanselman.com/blog/git-is-casesensitive-and-your-filesystem-may-not-be-weird-folder-merging-on-windows
8. Non case-sensitive Mac OS Extended formatting colliding with git clone | My Cellar Door, accessed February 17, 2026, https://javigon.com/2013/09/04/non-case-sensitive-mac-os-extended-colliding-with-git-clone/
9. How to make Git detect filename case changes: A Quick Fix - YouTube, accessed February 17, 2026, https://www.youtube.com/watch?v=x7-SGiDYD2k
10. Git: How to change the case of filenames - Adam Johnson, accessed February 17, 2026, https://adamj.eu/tech/2022/12/09/git-change-case-of-filenames/
11. How to clean the history on git repo? - Stack Overflow, accessed February 17, 2026, https://stackoverflow.com/questions/48186380/how-to-clean-the-history-on-git-repo
12. git case-insensitive directory renamed - Stack Overflow, accessed February 17, 2026, https://stackoverflow.com/questions/65618965/git-case-insensitive-directory-renamed